BACHELOR THESIS · TECHNICAL SUMMARY · RUPRECHT-KARLS-UNIVERSITY HEIDELBERG

Neural Decompilation with Program-Wide and Homogenized Context

Frederik Graewert · Artificial Intelligence for Programming Group
Supervisor: Prof. Dr. Artur Andrzejak · March 2026

`github.com/freddy913/Neural-Decompiler`

Compilers throw away names, types and high-level structure; a decompiler has to rebuild them from what is left. This thesis shows that for a neural decompiler **what the model is shown matters more than how large it is**: a 250M-parameter model with program-wide context beats a 13B-parameter system on structural validity, and context alone nearly doubles its functional correctness.

19.5%
Pass@1 with context (11.4% without)

37.1%
KLEE coverage – highest in benchmark

250M
parameters, vs. 13B baseline

~1M
generated assembly-to-C pairs

The problem

One instruction fits many source constructs: `mov [rbp-0x4], 0x5` may be a local assignment, a pointer write or an array store. Classical tools (Ghidra, Hex-Rays) resolve this with fixed heuristics and emit code that runs but reads badly – generic names, pointer arithmetic, types like `undefined4`.

Neural decompilers produce far more idiomatic C, but translate **one function at a time**. Everything defined elsewhere in the program is invisible, so the model invents the types, parameters and calls it cannot see.

> **Hypothesis.** To resolve structural ambiguity, program-wide context and data homogenization matter more than scaling model parameters.

What the model sees

Every training example is a flat text pair: the target function plus a bounded, scored slice of its neighbourhood.

```
HEADER:  #include <stdio.h>
TARGET:  func; endbr64; push rbp;
         mov edi, STRx402010; call <FUNC>; … ret
CALLERS: none
CALLEES: FN{ …add_numbers… }
         FN{ …multiply_numbers… }
```

Internal calls outside the provided context collapse to `<FUNC>`, so the model cannot rely on names it was never given. The header block is derived from external symbols found in the binary.

The pipeline

1 Recover structure
angr rebuilds the control-flow graph by emulation – correct function boundaries even under tail calls, fall-throughs and PLT stubs. The same graph yields the call graph.

2 Select context
Breadth-first search over the call graph; candidates scored on proximity, shared symbols and token cost, packed greedily into a budget proportional to the target's length.

3 Homogenize
Strip operand sizes (≈15% shorter), pool `.rodata` constants into placeholders, relative jump labels, resolve PLT symbols – identical placeholders in input and label.

4 Train
LongT5-base, 250M params, 8192-token window. A100 nodes on bwUniCluster with DeepSpeed ZeRO-3; fully resumable runs at a constant LR of 2e-5.

Results · HumanEval-C, N = 149, execution-verified

Generated C is compiled and run against unit tests (Pass@1). KLEE coverage = how much of the output a symbolic execution engine can traverse: structural validity, independent of logic.

MODEL / SETUP	PARAMS	COMPILES	PASS@1	SIM.	KLEE	GCOV
SK2Decompiler	≈13B	0.94	**0.79**	**0.62**	30.6%	84.1%
LLM4Decompile	6.7B	0.69	0.69	–	–	–
Idioms	2B	0.72	0.57	0.57	28.1%	43.3%
Ghidra + reasoning LLM	large	0.63	0.55	0.34	28.8%	**86.6%**
Ours · with context	**0.25B**	**0.44**	**0.20**	**0.60**	**37.1%**	**62.6%**
Ours · no context	**0.25B**	**0.40**	**0.11**	**0.59**	**29.6%**	**70.1%**

Sim. = Levenshtein similarity. Blue = best in column. Note the parameter disparity: the last two rows are 50× smaller than the first.

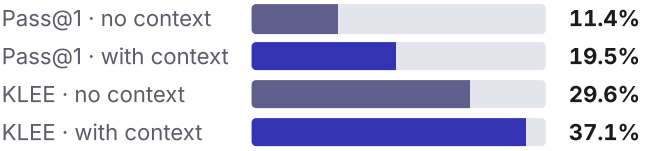
Reading it honestly

Where the small model loses: raw correctness. 0.20 Pass@1 against 0.79 is not close; that gap scales with parameters and pre-training.

Where it wins: structure. 37.1% KLEE coverage leads the field – the generated C has traversable, well-formed control flow instead of infeasible branches or endless loops. Similarity (0.60) sits just under the 13B model.

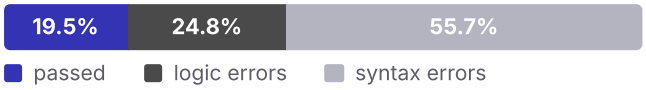
The Ghidra + LLM row is telling: highest line coverage, lowest similarity – a reasoning model *rewrites* the program. In reverse engineering the original implementation is the object of interest, so recovered structure beats a clean rewrite.

Ablation · same weights, context on vs. off



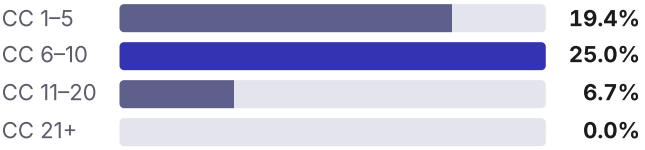
Training data mixed context-rich and context-free samples (leaf functions have no neighbours), so the test-set gap shows the model actively uses injected neighbours rather than ignoring them.

Failure analysis



A quarter of all outputs **compile and lie**: valid C that fails the tests. Several sit above 0.7 Levenshtein similarity to the ground truth – one inverted operator (`==` for `!=`) is a negligible textual difference and a total functional divergence. **BLEU-style metrics cannot see this; execution-based evaluation is a requirement, not a refinement.**

Complexity, not size



Pass@1 by cyclomatic complexity. Basic-block counts of passed and failed functions overlap almost entirely: long linear code is fine, deeply nested logic is not. Decoding matters too – a tuned repetition penalty (1.2, `no_repeat_ngram_size 5`) fixes a class of logic errors without retraining.

Limitations

- Deeply nested control flow (CC ≥ 21) fails completely – likely an architectural coherence limit, not a data problem.
- Structure is recovered; naming is not. Identifiers stay hallucinated.
- Fragmented cluster access forced a constant learning rate and split runs; some convergence was left on the table.
- Evaluated on HumanEval-C at `-O0`. Optimized, stripped or obfuscated real-world binaries are an untested, harder regime.

Where this goes next

- Reinforcement learning with compiler feedback and KLEE traces as reward – optimize for execution validity, not token prediction.
- Neuro-symbolic pipeline: this model emits a sound skeleton, a reasoning model repairs the semantics – the measured strengths are complementary.
- AST-constrained decoding to remove the 55.7% syntax-error class outright.
- Learned context selection and a denoising objective from optimized assembly to unoptimized source.

Open source. Full pipeline, training scripts and a one-click Streamlit demo – upload an ELF binary or point it at a GitHub repo, browse the discovered functions, toggle the context heuristic and compare outputs side by side.

`github.com/freddy913/Neural-Decompiler`